HireMind: Efficient CV–Job Fit Classification through Distillation and Quantization

Milestones 1 & 2 — Related Work, Dataset, Architecture, Baseline and Experimental Plan

Rawan Khojah **Hams Aljohani**
Effat University
CS4085 Deep Learning

Abstract

We study whether a 4.4M-parameter BERT-Tiny classifier for CV–job fit (*No Fit*, *Potential Fit*, *Good Fit*) improves when trained with knowledge distillation from a BERT-base teacher, and whether that improvement survives 8-bit post-training quantization that also covers the embedding table. This document covers Milestones 1 and 2: nine related papers, a dataset audit with a job-grouped split, the M0 architecture with parameter and MAC counts, a trained and checked M0 baseline, and the plan for M1, M2 and the ablations. The test set has not been used.

1 Introduction

Problem. HireMind, our Senior project, is a recruitment platform that scores applicants against job postings. Scoring every CV–job pair with a large external language model is slow, costs money per request, and sends personal data to a third party. A small model that runs locally on a CPU could provide one piece of evidence for the scoring step; it does not replace skill verification or the recruiter's decision.

Why it matters. Compact models are cheap to run but usually lose accuracy. Two standard remedies pull in opposite directions: knowledge distillation can make a small model more accurate, while quantization makes it cheaper but may cost accuracy. Whether a small accuracy gain from distillation survives quantization is what decides whether the extra training effort is worth it for deployment.

Research question. *Can teacher-guided training improve a compact CV–job fit classifier, and can post-training quantization reduce its storage while keeping that improvement?*

We compare three models: **M0**, BERT-Tiny fine-tuned with cross-entropy; **M1**, the same student trained with distillation from a fine-tuned BERT-base teacher; and **M2**, M1 after post-training INT8 quantization of its Linear layers and 8-bit quantization of its embeddings.

2 Related Work

Recruitment screening. Gan et al. (2024) build an LLM-agent pipeline for resume screening; their sentence classifier based on LLaMA2-7B reaches 87.73 F1 against 86.67 for BERT-Large. It shows the task can be automated but relies on very large models.

Architecture. BERT (Devlin et al., 2019) pretrains a bidirectional Transformer encoder that is fine-tuned per task; BERT-base reaches 84.6% on MNLI-m. Turc et al. (2019) release 24 compact BERT models, including BERT-Tiny (2 layers, hidden size 128, 4.4M parameters), and show that pretraining the student before distillation helps: a 6-layer student's GLUE development score rises from 82.8 to 84.4.

Distillation. Hinton et al. (2015) train a student on the teacher's temperature-softened probabilities; on MNIST, a distilled small network makes 74 errors versus 146 when trained on hard labels. DistilBERT (Sanh et al., 2019) distils BERT during pretraining into a 6-layer model that is 40% smaller and 60% faster while keeping 97% of its language understanding. TinyBERT (Jiao et al., 2020) adds layer-wise distillation of attention and hidden states; its 4-layer model reaches 96.8% of BERT-base on GLUE while being 7.5× smaller and 9.4× faster.

Quantization. Q8BERT (Zafrir et al., 2019) applies 8-bit quantization-aware training to BERT's embeddings and Linear layers, keeping MRPC F1 at 89.56 versus 90.0 in FP32, while post-training dynamic quantization drops to 87.88. Bondarenko et al. (2021) show why post-training quantization of BERT is hard: structured outliers in residual activations make naive W8A8 quantization fall from 83.06 to 71.03 GLUE points, which their per-embedding-group method restores to 82.45.

TernaryBERT (Zhang et al., 2020) combines both ideas: it uses distillation *during* quantization-aware training to reach 2-bit weights and embeddings, losing 1.3 GLUE points (78.2 to 76.9) while being 14.9× smaller.

2.1 Research gap

What existing papers show. Distillation helps compact BERT students (Turc et al., 2019; Sanh et al., 2019; Jiao et al., 2020); 8-bit quantization can keep BERT accuracy, more easily with training (Zafrir et al., 2019) than post-training (Bondarenko et al., 2021); and distillation can be used to recover the loss caused by quantization (Zhang et al., 2020).

What is still unclear. These studies use general benchmarks and mostly larger models. They do not separately measure whether a distillation gain obtained *before* quantization survives *post-training* 8-bit quantization when the embeddings—90.6% of BERT-Tiny's weights—are quantized too, nor do they evaluate CV–job pair classification. We do not claim that this combination has never been studied; we claim it is insufficiently evaluated for this setting.

How our experiments answer it. Matched comparisons M1 vs. M0 (does distillation help?), M2 vs. M1 (what does quantization cost?) and M2 vs. M0 (is the gain kept?), plus ablations that quantize the embeddings and the Linear layers separately.

3 Dataset

Source and licence. We use the resume–job-description fit dataset on Hugging Face[1], pinned to revision `08978e2` and verified by SHA-256 file hashes. It has no dataset card and no licence; the instructor approved it for this course project. We do not redistribute CV text: only hashed IDs are stored in our repository. Its source and labelling method are undocumented, so all results are interpreted as agreement with these benchmark labels, not as hiring validity.

Size and classes. Each row is a (CV, job description, label) triple. Table 2 summarises the audit; Figure 1 shows class counts and input lengths.

Class imbalance. *No Fit* is half of the data and the other two classes a quarter each. This is moderate, so we keep the natural distribution and use macro-F1, which weights the three classes equally, as the main metric.

[1] https://huggingface.co/datasets/cnamuangtoun/resume-job-description-fit

Duplicates and leakage. The same CV is paired with many jobs, and the official files share 476 CVs but no job descriptions. A row-level random split would therefore place the same job in training and evaluation. We split by *job*, so every evaluation job is unseen; CVs can still recur, so our results measure generalisation to new jobs, not to new candidates. Near-duplicate job descriptions are merged into one group before splitting.

Preprocessing and splitting. The policy was fixed before any model was trained: (1) CV and job IDs are SHA-256 hashes of whitespace-normalised, lower-cased text; (2) all rows of the six conflicting pairs are removed; (3) exact duplicate pairs keep one copy, and a training copy of a pair that also appears in test is dropped; (4) training rows whose job is a near-duplicate of a test job are removed (73 rows); (5) the official test file is kept as the test set; (6) validation is one fold of a stratified group 5-fold split of the cleaned training file (groups = job, stratified by label, seed 42). Text is tokenised as `[CLS] CV [SEP] job [SEP]`, truncated longest-first to 256 tokens and padded. Every pair is longer than this limit (median 1563 word-pieces: CV median 1,043, job 422), so the model sees roughly the first 125 tokens of each text; this is a known limitation shared by all three models. The exact row IDs of each split are saved in `splits/splits.csv`. The test set is only for the final evaluation.

Label check. Because the labelling method is unknown, both authors will independently label 90 training pairs (30 per class, seed 42) with a written rubric, without seeing the dataset labels, and report Cohen's κ and agreement with the dataset. The sheet is generated by our notebook and kept private.

4 Methodology

4.1 M0 architecture

Figure 2 shows M0. **Input:** a CV and a job description, tokenised into one sequence of 256 word-pieces with segment IDs 0 (CV) and 1 (job). **Embeddings:** each token gets a 128-dimensional vector (word + position + segment), normalised with LayerNorm. **Encoder:** two Transformer layers; in each, two self-attention heads let every CV token attend to every job token and vice versa, followed by a feed-forward network (128→512→128, GELU), with residual connections and LayerNorm. **Output:** the final `[CLS]` vector passes through the pooler (128→128, tanh) and a new Linear layer (128→3); softmax gives the three class probabilities.

Paper	Main idea	Model	Dataset	Results	Limitations (for us)
Gan et al. (2024)	LLM agents for resume screening: sentence classification, grading, decision	LLaMA2-7B/13B, GPT-3.5	1,000 IT resumes	87.73 F1 (vs. 86.67 BERT-Large) on sentence classification	Very large models; different task; no cost analysis
Devlin et al. (2019)	Bidirectional Transformer pretraining, then task fine-tuning	BERT-base / large	GLUE, SQuAD	MNLI-m 84.6% (base)	General benchmarks; costly; 512-token limit
Turc et al. (2019)	Pretrain compact students, then distil	24 compact BERTs incl. BERT-Tiny	GLUE, Amazon reviews	6-layer: 82.8 → 84.4 GLUE dev	Tiny models not studied on matching tasks; no quantization
Hinton et al. (2015)	Train student on softened teacher probabilities	MLPs, speech DNNs	MNIST, speech	146 → 74 MNIST errors	Not Transformers or text pairs
Sanh et al. (2019)	Distil BERT during pretraining (triple loss)	6-layer DistilBERT	GLUE, IMDb, SQuAD	40% smaller, 60% faster, 97% of BERT	Student still 66M params; no quantization
Jiao et al. (2020)	Layer-wise (attention, hidden) distillation	4-/6-layer TinyBERT	GLUE	96.8% of BERT-base, 7.5× smaller, 9.4× faster	Needs data augmentation and a two-stage pipeline
Zafrir et al. (2019)	8-bit quantization-aware training of BERT	BERT-base / large	GLUE, SQuAD	MRPC F1 90.0 FP32, 89.56 QAT, 87.88 PTQ	Requires retraining; full-size BERT only
Bondarenko et al. (2021)	Explain and fix PTQ failures (activation outliers)	BERT-base	GLUE	W8A8 PTQ 71.03 → 82.45 (FP32 83.06)	Full-size BERT; no distillation
Zhang et al. (2020)	Distillation-aware 2-bit quantization	BERT-base	GLUE, SQuAD	14.9× smaller; GLUE 78.2 → 76.9	Quantization-aware training; distillation used to recover loss, not studied as a separate gain

Table 1: Comparison of the nine papers. Results are those reported by the authors on their own benchmarks.

Rows: total (train / test file)	8000 (6241 / 1759)
Classes: No / Potential / Good Fit	4,000 / 2,000 / 2,000
Unique CVs / job descriptions	643 / 351
Pairs with conflicting labels (rows)	6 (12)
Extra rows from exact duplicate pairs	7
Empty fields	0
CVs in both train and test files	476
Jobs in both train and test files	0
Near-duplicate job pairs, cos ≥ 0.95 (across files)	3 (2)
Near-duplicate CV pairs, cos ≥ 0.95	23
Pair length, median / 90th pct. (tokens)	1563 / 2467
Pairs longer than 256 tokens	100.0%

Table 2: Dataset audit before cleaning.

Split	Pairs	Jobs	No	Pot.	Good
Train	4993	223	2533	1231	1229
Validation	1162	55	579	294	289
Test	1759	71	857	444	458

Table 3: Final splits. No job group appears in more than one split.

Why it suits the task. Fit depends on matching requirements in the job to evidence in the CV. A cross-encoder lets attention compare the two texts token by token, which bag-of-words features cannot do. BERT-Tiny keeps this mechanism at 4.4M parameters, small enough for CPU inference, and is the compact student released by Turc et al. (2019). We use its **pretrained** weights; only the classifier head is new.

Parameters and MACs. Table 4 gives exact parameter counts and MACs for one pair of 256 tokens. Per layer, the Q/K/V/O projections cost $4LH^2$, the attention scores and weighted sum $2L^2H$, and the feed-forward network $2LHI$ MACs (L=256, H=128, I=512), i.e. 67.1M per layer. The embeddings hold 90.6% of the parameters but cost almost no computation, which is why quantizing them matters for storage but not for speed. The notebook re-measures MACs with PyTorch's `FlopCounterMode`: 134.2 M. For comparison, a BERT-base teacher needs about 22.95G MACs per pair, 171× more.

4.2 Baseline implementation and results

Training setup. AdamW (weight decay 0.01), batch size 16, 3 epochs (939 updates), linear schedule with 10% warm-up, gradient clipping at 1.0, full FP32 precision. We tried four learning rates $\{2 \cdot 10^{-5}, 5 \cdot 10^{-5}, 10^{-4}, 3 \cdot 10^{-4}\}$ with seed 42, evaluated on validation every 100 steps, and kept

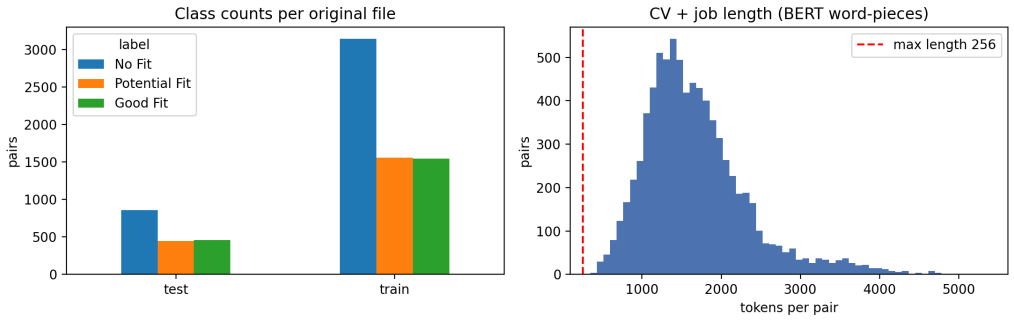


Figure 1: Left: class counts in the original train and test files. Right: CV + job length in word-pieces; the red line is our 256-token limit.

Block	Parameters	MACs
Embeddings	3,972,864	≈0
Encoder layer 1	198,272	67.1M
Encoder layer 2	198,272	67.1M
Pooler	16,512	16.4K
Classifier	387	0.4K
Total	4,386,307	134.2M

Table 4: M0 cost per block (one CV–job pair, 256 tokens).

Model (validation)	Macro-F1	Accuracy
Majority class	22.2	49.8
TF-IDF + logistic regression	36.7	52.8
M0, seed 42 (lr 0.0003)	48.2	56.2
M0, 3 seeds (mean ± s.d.)	44.6 ± 5.0	53.5 ± 2.4

LR	Macro-F1	Acc.	Val. loss	Best step
2e-05	25.8	50.9	1.007	900
5e-05	42.6	53.5	0.978	939
0.0001	42.3	53.3	0.999	939
0.0003	48.2	56.2	1.113	939

Table 5: Top: M0 against simple references. Bottom: learning-rate search (seed 42). All numbers are on the validation split, in %.

the checkpoint with the best validation macro-F1. The selected learning rate was then rerun with seeds 43 and 44.

Overfit-one-batch test. With dropout disabled, M0 was trained on one fixed batch of 16 pairs for 300 steps (learning rate 10^{-3}). The loss fell from 1.150 to 0.0009 and training accuracy reached 100%, so the test passed (criterion: loss < 0.05 and 100% accuracy).

Results. Table 5 compares M0 with two simple references fitted on the same training split. Figure 3 shows the loss curves.

Is the baseline credible? M0 clearly beats both simple references, passes the overfit test, and its training loss decreases steadily (Figure 3). Three observations shape the next steps. (1) After about 200 updates the validation loss rises while the training loss keeps falling, yet validation macro-F1 continues to improve: the model becomes more confident without becoming less accurate. Selecting checkpoints by macro-F1 rather than by loss is therefore important, and M1 will use the same rule. (2) The best learning rate is the largest one tried, and macro-F1 varies across seeds (44.6 ± 5.0). Paired seeds are essential for E1: a single run could show or hide a 1-point difference by chance. (3) 100.0% of pairs are longer than 256 tokens (median 1563), so the model sees only the beginning of each CV and job description, often contact details and company introductions. *No Fit* is recognised best, while *Potential* and *Good Fit* are often confused with each other and with *No Fit*. The limit is the same for M0, M1 and M2, so the comparisons stay fair, but it caps absolute performance; selecting the skills and requirements sections is a candidate follow-up experiment.

Computational cost. Saved weights (`state_dict`): 16.74 MiB. CPU latency (Intel(R) Xeon(R) CPU @ 2.00GHz, 1 thread, batch 1, 256 tokens, 30 warm-up and 200 timed runs): median 5.44 ms, p95 8.79 ms for the model alone and 24.18 ms including tokenisation. Peak process memory: 864 MiB, of which 771 MiB is PyTorch and Transformers after import, so loading and running the model adds about 94 MiB. Tokenising the long raw texts takes longer than running the model itself. Training the selected run took 16 s on Tesla T4 with a peak GPU memory of 492 MiB.

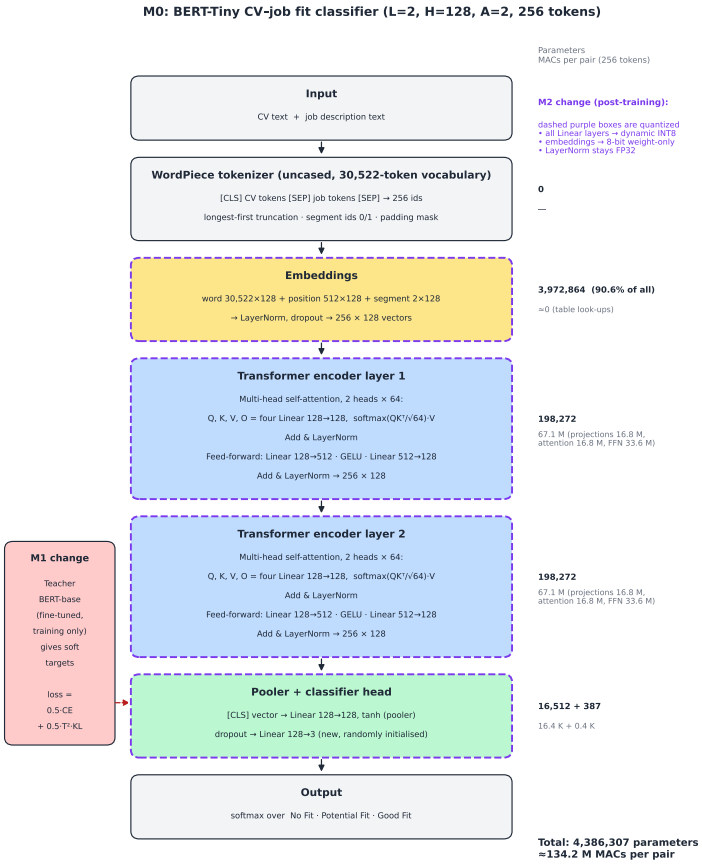


Figure 2: M0 with parameters and MACs per block. Dashed purple blocks are the modules quantized in M2; the red box shows the extra distillation loss used to train M1.

4.3 Proposed M1: distillation (Part 1)

Change. Same student, same initialisation, same data and update budget as M0, but trained with

$$L = 0.5\, L_{CE} + 0.5\, T^2\, \mathrm{KL}\big(\sigma(z_T/T) \,\|\, \sigma(z_S/T)\big), \quad (1)$$

where z_T, z_S are teacher and student logits and σ is softmax. The teacher is `bert-base-uncased` with a three-class head, fine-tuned on the same training split with the same input format, its learning rate ($2\cdot10^{-5}$ or $5\cdot10^{-5}$) and checkpoint chosen on validation macro-F1. Its logits for the training pairs are computed once in evaluation mode and cached, so every M1 run uses the same targets. M1 tries learning rates $\{5\cdot10^{-5}, 10^{-4}\} \times T \in \{2, 4\}$, four trials, the same number as M0.

Why it should help. Hard labels say only which class is right. The teacher's softened probabilities also say how close the other classes are—for example that a pair is *Good Fit* but close to *Potential Fit*. This extra signal acts as a regulariser for a small model trained on limited data, which is the setting where Hinton et al. (2015) and Turc et al. (2019) report gains. It may fail if the teacher is not clearly better than M0 on validation, which we will check before running M1.

4.4 Proposed M2: quantization (Part 2)

Change. A CPU copy of the selected M1 converted with PyTorch `quantize_dynamic`: Linear layers use dynamic INT8 (weights stored in 8 bits, activation scales computed per input at run time) and embeddings use 8-bit weight-only quantization with per-row scales; LayerNorm stays FP32. No retraining.

Why it reduces cost. Storing a weight in 1 byte instead of 4 shrinks the 90.6% of parameters in the embeddings about fourfold; we therefore expect M2's saved size to be well below 40% of M1's (our H2 target), with per-row scale metadata and FP32 LayerNorms as the remainder. INT8 matrix multiplication can also speed up the Linear layers, but for a model this small the conversion overhead may cancel the gain, so latency is measured, not assumed. MAC counts do not change.

Possible quality loss. Rounding errors in rare-

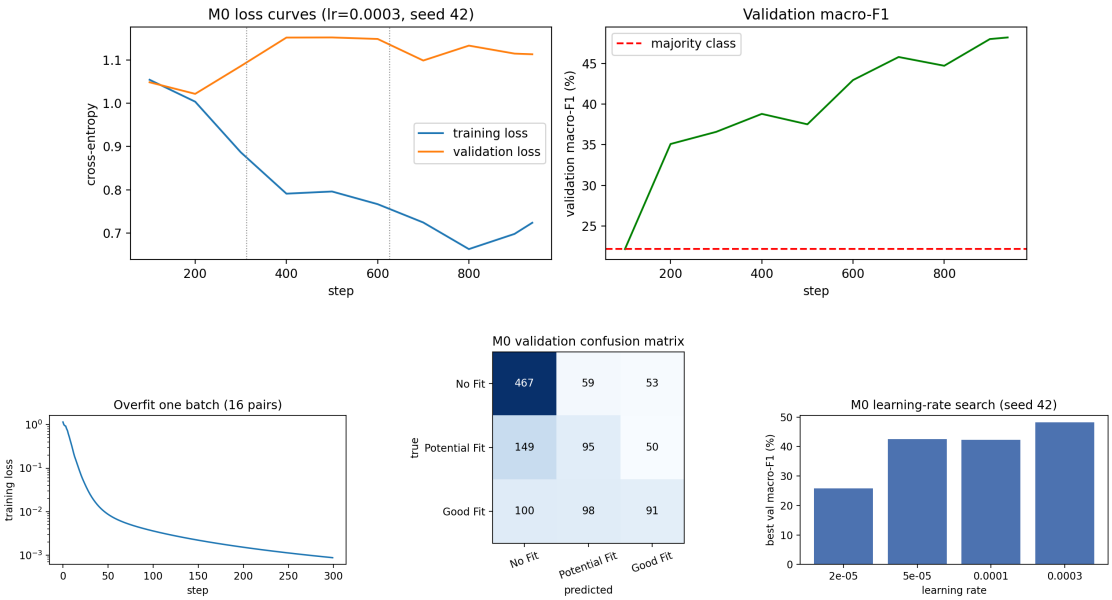


Figure 3: Top: M0 training and validation loss and validation macro-F1 (dotted lines mark epochs). Bottom: overfit-one-batch test, validation confusion matrix, learning-rate search.

	Comparison	Question
E1	M1 vs. M0	Does distillation help? (H1: $F_1 - F_0 \geq 1.0$)
E2	M2 vs. M1	How much storage is saved and what quality is lost? (H2: $S_2 \leq 0.40 S_1$, $F_1 - F_2 \leq 1.0$)
E3	M2 vs. M0	Is the gain kept? (H2: $F_2 - F_0 \geq 0.5(F_1 - F_0)$)
E4a	M1 + Linear-only INT8 vs. embeddings-only vs. both	Which quantized component explains the size saving and the quality loss?
E4b	M1 with T=1 vs. selected T	Does softening the teacher's probabilities matter?
E4c	M0 + same quantization	Is a distilled model more or less robust to quantization than M0? (optional)

Table 6: Planned experiments. F_i: mean test macro-F1 over three seeds; S_i: saved size.

word embeddings and in Linear weights. Dynamic quantization computes activation ranges per input, which avoids the fixed-range outlier problem reported by Bondarenko et al. (2021) for static post-training quantization, but a small model has little redundancy to absorb errors.

4.5 Experimental plan

Table 6 lists the experiments. **Fairness:** M0 and M1 share the split, tokenisation, 256-token limit, pretrained initialisation and head seed, optimiser, update budget and number of tuning trials; only the loss differs. M2 differs from M1 only by quantization. **Repeats:** the selected settings run with seeds 42, 43 and 44; we report each paired difference and mean $\pm$ s.d., without significance claims. **Test set:** all choices are frozen on validation before the test set is evaluated once. If embedding quantization fails, H2 is reported as untestable for the planned M2 and the Linear-only variant as exploratory.

4.6 Metrics

Quality: macro-F1 (main), accuracy, per-class precision/recall/F1 and confusion matrices. **Cost:** parameter count, saved `state_dict` size (main compression measure, tokenizer and config excluded), MACs on the model graph, CPU latency (model-only and with tokenisation; median and p95) and peak process memory, all measured in a fresh process under the conditions above. We also record teacher and student training time.

4.7 Environment and reproducibility

Training runs on Google Colab (Tesla T4), CPU measurements on Intel(R) Xeon(R) CPU @ 2.00GHz, with PyTorch 2.11.0+cu130 and Transformers 5.20.1. Seeds are fixed (Python, NumPy, PyTorch, data loader), the dataset revision and file hashes are pinned, the split IDs are saved, and the whole pipeline is one notebook in our repository[2].

[2] https://github.com/Hams770/HireMind-DL

Next steps. Fine-tune and check the teacher, run M1 (E1, E4b), quantize (E2, E3, E4a), and complete the label check.

AI use

ChatGPT and Claude assisted with literature search and summaries, code for the data audit, notebook and figures, and drafting. The authors checked the content, ran the experiments and are responsible for this document.

References

Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. 2021. Understanding and Overcoming the Challenges of Efficient Transformer Quantization. *EMNLP*, pages 7947–7969. DOI: 10.18653/v1/2021.emnlp-main.627.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*, pages 4171–4186.

Chengguang Gan, Qinghao Zhang, and Tatsunori Mori. 2024. Application of LLM Agents in Recruitment: A Novel Framework for Automated Resume Screening. *Journal of Information Processing*, 32:881–893. DOI: 10.2197/ipsjjip.32.881.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2020. TinyBERT: Distilling BERT for Natural Language Understanding. *Findings of EMNLP*, pages 4163–4174. DOI: 10.18653/v1/2020.findings-emnlp.372.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *EMC*^{2} *Workshop at NeurIPS*. arXiv:1910.01108.

Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Well-Read Students Learn Better: On the Importance of Pre-training Compact Models. *arXiv:1908.08962*.

Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8BERT: Quantized 8Bit BERT. *EMC*^{2} *Workshop at NeurIPS*. DOI: 10.1109/EMC2-NIPS53020.2019.00016.

Wei Zhang, Lu Hou, Yichun Yin, Lifeng Shang, Xiao Chen, Xin Jiang, and Qun Liu. 2020. TernaryBERT: Distillation-aware Ultra-low Bit BERT. *EMNLP*, pages 509–521. DOI: 10.18653/v1/2020.emnlp-main.37.